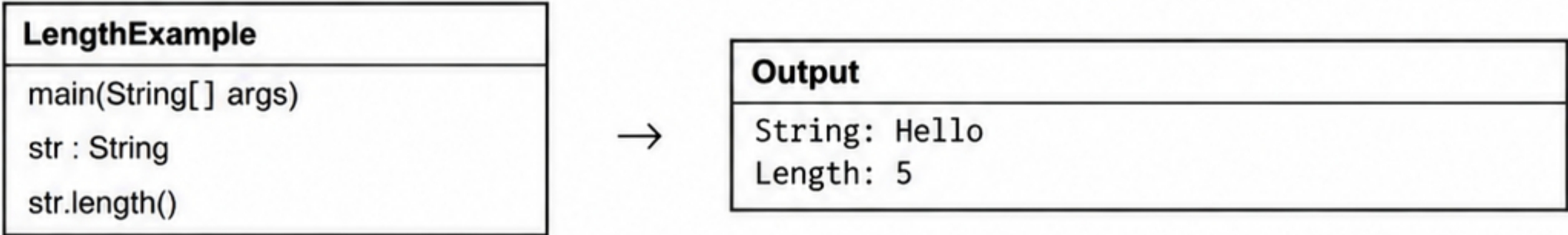


1. length() — str

Objective	
Objective: To understand and demonstrate the Java String method length() using a simple example.	
Task Description	
Task Description: Write and execute the class LengthExample to demonstrate that length() finds the number of characters in a string.	
Step-wise Execution 1. Create a String variable named str. 2. Store the value "Hello" in it. 3. Call the length() method using str. 4. Display the returned number of characters.	Algorithm / Logic 1. Start. 2. Initialize str with "Hello". 3. Find the length using str.length(). 4. Print the string and length. 5. Stop.

UML Diagram



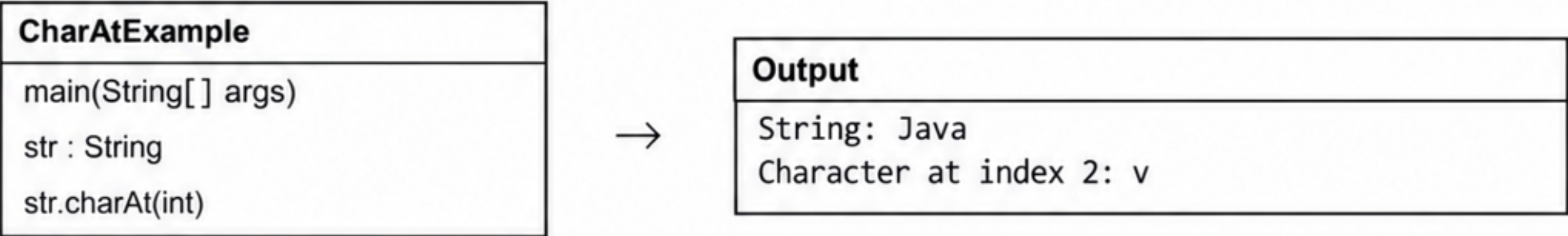
Program Code
<pre>class LengthExample { public static void main(String[] args) { String str = "Hello"; System.out.println("String: " + str); System.out.println("Length: " + str.length()); } }</pre>

Code Explanation	Output Screenshot
1. `String str = "Hello";` creates a String variable containing the text "Hello". 2. `str.length()` returns the total number of characters in the string. 3. `System.out.println()` displays the string and the calculated length.	

2. charAt() — str

Objective	
Objective: To understand and demonstrate the Java String method charAt() using a simple example.	
Task Description	
Task Description: Write and execute the class CharAtExample to demonstrate that charAt() returns the character at a specified index position.	
Step-wise Execution 1. Create a String variable named str. 2. Store the value "Java" in it. 3. Call the charAt(2) method using str. 4. Display the returned character.	Algorithm / Logic 1. Start. 2. Initialize str with "Java". 3. Access the character at index 2 using str.charAt(2). 4. Print the string and the selected character. 5. Stop.

UML Diagram



Program Code
<pre>class CharAtExample { public static void main(String[] args) { String str = "Java"; System.out.println("String: " + str); System.out.println("Character at index 2: " + str.charAt(2)); } }</pre>

Code Explanation	Output Screenshot
1. <code>`String str = "Java";`</code> creates a String variable containing the text "Java". 2. <code>`str.charAt(2)`</code> returns the character at index 2 in the string. 3. <code>`System.out.println()`</code> displays the string and the selected character.	

3. concat() — str1, str2

Objective

Objective: To understand and demonstrate the Java String method **concat()** using a simple example.

Task Description

Task Description: Write and execute the class **ConcatExample** to demonstrate that concat() joins two strings.

Step-wise Execution

1. Create two String variables str1 and str2.
2. Store the value "Hello " in str1.
3. Store the value "World" in str2.
4. Call the concat() method using str1.
5. Store the result in a variable result.
6. Display the first string, second string and the concatenated result.

Algorithm / Logic

1. Start.
2. Initialize str1 with "Hello " and str2 with "World".
3. Concatenate the strings using str1.concat(str2).
4. Store the result in variable result.
5. Print the first string, second string and the result.
6. Stop.

UML Diagram

ConcatExample

main(String[] args)
str1 : String
str2 : String
result : String
str1.concat(str2)



Output

First String: Hello
Second String: World
Result: Hello World

Program Code

```
class ConcatExample {  
    public static void main(String[] args) {  
        String str1 = "Hello ";  
        String str2 = "World";  
  
        String result = str1.concat(str2);  
  
        System.out.println("First String: " + str1);  
        System.out.println("Second String: " + str2);  
        System.out.println("Result: " + result);  
    }  
}
```

Code Explanation

1. `String str1 = "Hello ";` and `String str2 = "World";` create two String variables.
2. `str1.concat(str2)` joins str2 to the end of str1.
3. The concatenated string is stored in `result`.
4. `System.out.println()` displays the first string, second string and the result.

Output Screenshot

First String: Hello
Second String: World
Result: Hello World

4. equals() — str1, str2

Objective

Objective: To understand and demonstrate the Java String method **equals()** using a simple example.

Task Description

Task Description: Write and execute the class **EqualsExample** to demonstrate that equals() compares two strings for exact equality.

Step-wise Execution

1. Create two String variables str1 and str2.

2. Store the value "Java" in str1.

3. Store the value "Java" in str2.

4. Call the equals() method using str1.

5. Store the result in a boolean variable result.

6. Display the first string, second string and the result.

Algorithm / Logic

1. Start.

2. Initialize str1 with "Java" and str2 with "Java".

3. Compare the strings using str1.equals(str2).

4. Store the returned value (true/false) in result.

5. Print the first string, second string and the result.

6. Stop.

UML Diagram

EqualsExample

main(String[] args)

str1 : String

str2 : String

result : boolean

str1.equals(str2)



Output

First String: Java

Second String: Java

Are they equal? true

Program Code

```
class EqualsExample {
    public static void main(String[] args) {
        String str1 = "Java";
        String str2 = "Java";

        boolean result = str1.equals(str2);

        System.out.println("First String: " + str1);
        System.out.println("Second String: " + str2);
        System.out.println("Are they equal? " + result);
    }
}
```

Code Explanation

1. `String str1 = "Java";` and `String str2 = "Java";` create two String variables with the same text.

2. `str1.equals(str2)` compares the contents of both strings.

3. The method returns `true` because both strings are exactly the same.

4. `System.out.println()` displays the first string, second string and the result.

Output Screenshot

First String: Java

Second String: Java

Are they equal? true

5. equalsIgnoreCase() — str1, str2

Objective	
Objective: To understand and demonstrate the Java String method equalsIgnoreCase() using a simple example.	
Task Description	
Task Description: Write and execute the class EqualsIgnoreCaseExample to demonstrate that equalsIgnoreCase() compares two strings ignoring case (uppercase and lowercase).	
Step-wise Execution	Algorithm / Logic
1. Create two String variables str1 and str2. 2. Store the value "Java" in str1. 3. Store the value "JAVA" in str2. 4. Call the equalsIgnoreCase() method using str1. 5. Store the result in a boolean variable result. 6. Display the first string, second string and the result.	1. Start. 2. Initialize str1 with "Java" and str2 with "JAVA". 3. Compare the strings using str1.equalsIgnoreCase(str2). 4. Store the returned value (true/false) in result. 5. Print the first string, second string and the result. 6. Stop.

UML Diagram

EqualsIgnoreCaseExample
main(String [] args) str1 : String str2 : String result : boolean str1.equalsIgnoreCase(str2)



Output
First String: Java Second String: JAVA Are they equal? true

Program Code
<pre>class EqualsIgnoreCaseExample { public static void main(String[] args) { String str1 = "Java"; String str2 = "JAVA"; boolean result = str1.equalsIgnoreCase(str2); System.out.println("First String: " + str1); System.out.println("Second String: " + str2); System.out.println("Are they equal? " + result); } }</pre>

Code Explanation	Output Screenshot
1. `String str1 = "Java";` and `String str2 = "JAVA";` create two String variables with the same text but different cases. 2. `str1.equalsIgnoreCase(str2)` compares the strings ignoring uppercase and lowercase. 3. The method returns `true` because both strings have the same letters, just in different cases. 4. `System.out.println()` displays the first string, second string and the result.	

6. toUpperCase() — str

Objective

Objective: To understand and demonstrate the Java String method **toUpperCase()** using a simple example.

Task Description

Task Description: Write and execute the class **UpperCaseExample** to demonstrate that toUpperCase() converts all letters in a string to a uppercase.

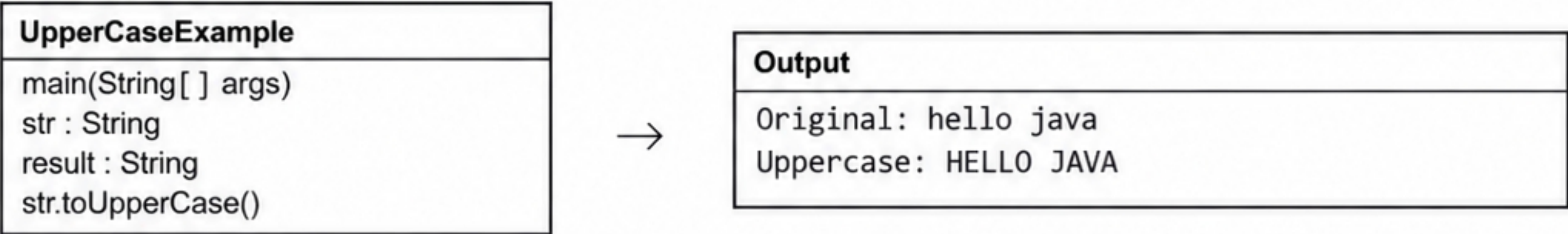
Step-wise Execution

1. Create a String variable named str.
2. Store the value "hello java" in it.
3. Call the toUpperCase() method using str.
4. Store the returned value in a variable result.
5. Display the original string and the uppercase string.

Algorithm / Logic

1. Start.
2. Initialize str with "hello java".
3. Apply toUpperCase() to str.
4. Store the result in result.
5. Print the original and uppercase strings.
6. Stop.

UML Diagram



Program Code

```
class UpperCaseExample {  
    public static void main(String[] args) {  
        String str = "hello java";  
  
        String result = str.toUpperCase();  
  
        System.out.println("Original: " + str);  
        System.out.println("Uppercase: " + result);  
    }  
}
```

Code Explanation

1. `String str = "hello java";` creates a String variable containing the text "hello java".
2. `str.toUpperCase()` converts all lowercase letters in the string to uppercase.
3. The converted string is stored in `result`.
4. `System.out.println()` displays the original string and the uppercase string.

Output Screenshot

Original: hello java
Uppercase: HELLO JAVA

7. toLowerCase() — str

Objective

Objective: To understand and demonstrate the Java String method **toLowerCase()** using a simple example.

Task Description

Task Description: Write and execute the class **LowerCaseExample** to demonstrate that toLowerCase() converts all letters in a string to lowercase.

Step-wise Execution

Algorithm / Logic

1. Create a String variable named str.

2. Store the value "HELLO JAVA" in it.

3. Call the toLowerCase() method using str.

4. Store the returned value in a variable result.

5. Display the original string and the lowercase string.

1. Start.

2. Initialize str with "HELLO JAVA".

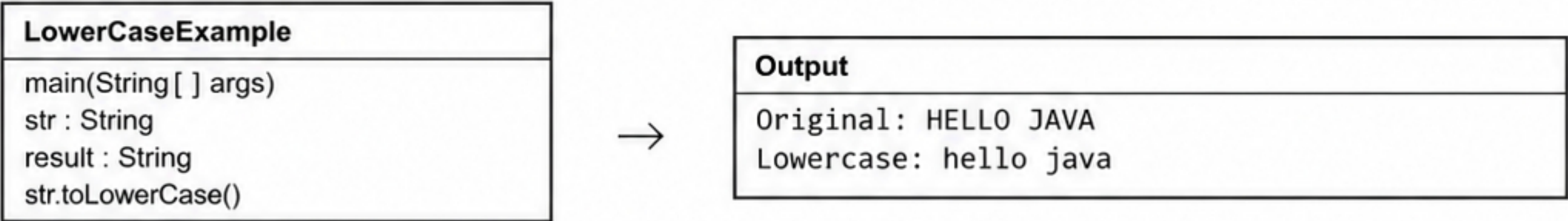
3. Apply toLowerCase() to str.

4. Store the result in result.

5. Print the original and lowercase strings.

6. Stop.

UML Diagram



Program Code

```
class LowerCaseExample {  
    public static void main(String[] args) {  
        String str = "HELLO JAVA";  
  
        String result = str.toLowerCase();  
  
        System.out.println("Original: " + str);  
        System.out.println("Lowercase: " + result);  
    }  
}
```

Code Explanation

Output Screenshot

1. `String str = "HELLO JAVA";` creates a String variable containing the text "HELLO JAVA".

2. `str.toLowerCase()` converts all uppercase letters in the string to lowercase.

3. The converted string is stored in `result`.

4. `System.out.println()` displays the original string and the lowercase string.

Original: HELLO JAVA

Lowercase: hello java

8. substring() — str

Objective

Objective: To understand and demonstrate the Java String method **substring()** using a simple example.

Task Description

Task Description: Write and execute the class **SubstringExample** to demonstrate that **substring()** returns a part of the string from the specified beginning index (inclusive) to ending index (exclusive).

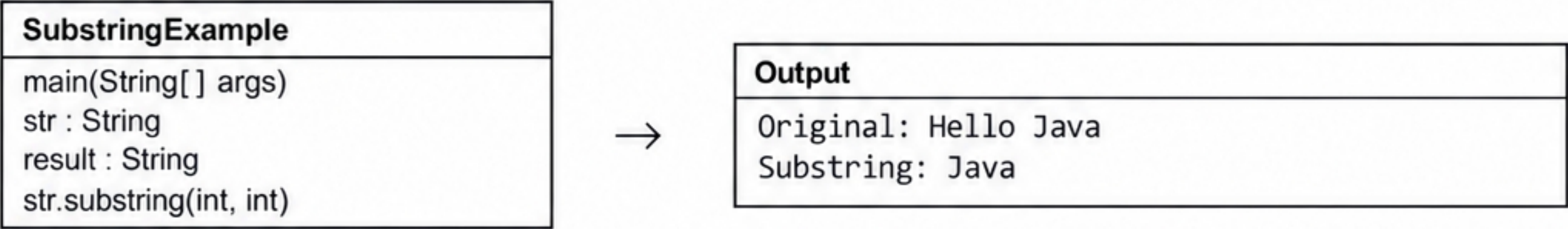
Step-wise Execution

1. Create a String variable named str.
2. Store the value "Hello Java" in it.
3. Call the substring(6, 10) method using str.
4. Store the returned value in a variable result.
5. Display the original string and the substring.

Algorithm / Logic

1. Start.
2. Initialize str with "Hello Java".
3. Extract a substring using str.substring(6, 10).
4. Store the result in result.
5. Print the original string and the substring.
6. Stop.

UML Diagram



Program Code

```
class SubstringExample {  
    public static void main(String[] args) {  
        String str = "Hello Java";  
  
        String result = str.substring(6, 10);  
  
        System.out.println("Original: " + str);  
        System.out.println("Substring: " + result);  
    }  
}
```

Code Explanation

1. `String str = "Hello Java";` creates a String variable containing the text "Hello Java".
2. `str.substring(6, 10)` returns the substring from index 6 (inclusive) to index 10 (exclusive).
3. The extracted substring is stored in `result`.
4. `System.out.println()` displays the original string and the substring.

Output Screenshot

Original: Hello Java
Substring: Java

9. indexOf() — str

Objective

Objective: To understand and demonstrate the Java String method **indexOf()** using a simple example.

Task Description

Task Description: Write and execute the class **IndexOfExample** to demonstrate that **indexOf()** returns the index position of a specified character in a string.

Step-wise Execution

Algorithm / Logic

1. Create a String variable named str.

2. Store the value "Hello Java" in it.

3. Call the indexOf('J') method using str.

4. Store the returned value in a variable result.

5. Display the string and the index position.

1. Start.

2. Initialize str with "Hello Java".

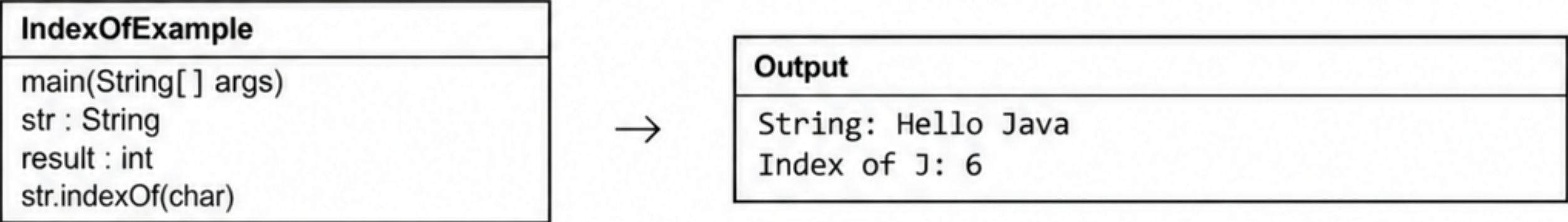
3. Find the index of character 'J' using str.indexOf('J').

4. Store the result in result.

5. Print the string and the index position.

6. Stop.

UML Diagram



Program Code

```
class IndexOfExample {  
    public static void main(String[] args) {  
        String str = "Hello Java";  
  
        int result = str.indexOf('J');  
  
        System.out.println("String: " + str);  
        System.out.println("Index of J: " + result);  
    }  
}
```

Code Explanation

Output Screenshot

1. ``String str = "Hello Java";`` creates a String variable containing the text "Hello Java".

2. ``str.indexOf('J')`` returns the index position of the first occurrence of the character 'J' in the string.

3. The index of 'J' in "Hello Java" is 6 (counting starts from 0).

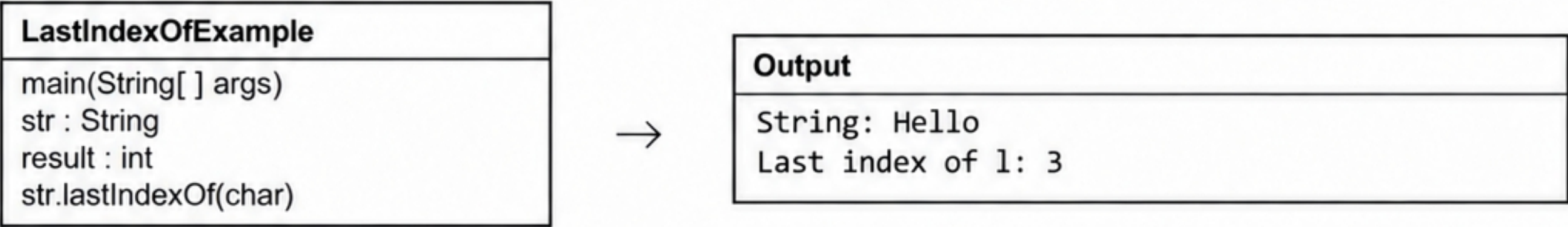
4. ``System.out.println()`` displays the string and the index position.

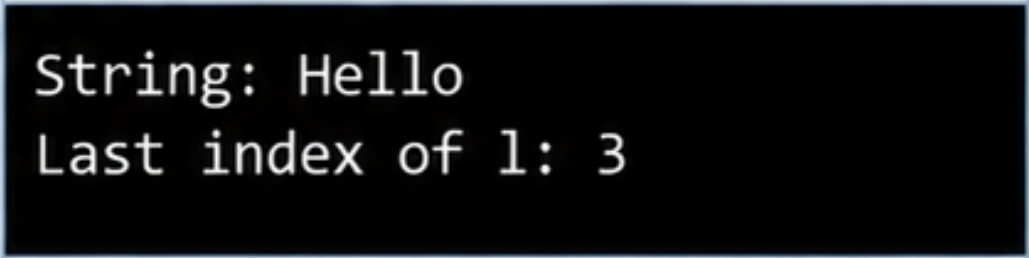
String: Hello Java
Index of J: 6

10. lastIndexOf() — str

Objective	
Objective: To understand and demonstrate the Java String method lastIndexOf() using a simple example.	
Task Description	
Task Description: Write and execute the class LastIndexOfExample to demonstrate that lastIndexOf() returns the index position of the last occurrence of a specified character in a string.	
Step-wise Execution	Algorithm / Logic
1. Create a String variable named str. 2. Store the value "Hello" in it. 3. Call the lastIndexOf('l') method using str. 4. Store the returned value in a variable result. 5. Display the string and the last index position.	1. Start. 2. Initialize str with "Hello". 3. Find the last index of character 'l' using str.lastIndexOf('l'). 4. Store the result in result. 5. Print the string and the last index position. 6. Stop.

UML Diagram



Program Code	
<pre>class LastIndexOfExample { public static void main(String[] args) { String str = "Hello"; int result = str.lastIndexOf('l'); System.out.println("String: " + str); System.out.println("Last index of l: " + result); } }</pre>	
Code Explanation	Output Screenshot
1. <code>String str = "Hello";</code> creates a String variable containing the text "Hello". 2. <code>str.lastIndexOf('l')</code> returns the index position of the last occurrence of the character 'l' in the string. 3. The last index of 'l' in "Hello" is 3 (counting starts from 0). 4. <code>System.out.println()</code> displays the string and the last index position.	

11. contains() — str

Objective

Objective: To understand and demonstrate the Java String method **contains()** using a simple example.

Task Description

Task Description: Write and execute the class **ContainsExample** to demonstrate that **contains()** checks whether a string contains a specified sequence of characters.

Step-wise Execution

Algorithm / Logic

1. Create a String variable named str.

2. Store the value "Hello Java" in it.

3. Call the contains("Java") method using str.

4. Store the returned value in a boolean variable result.

5. Display the string and whether it contains "Java".

1. Start.

2. Initialize str with "Hello Java".

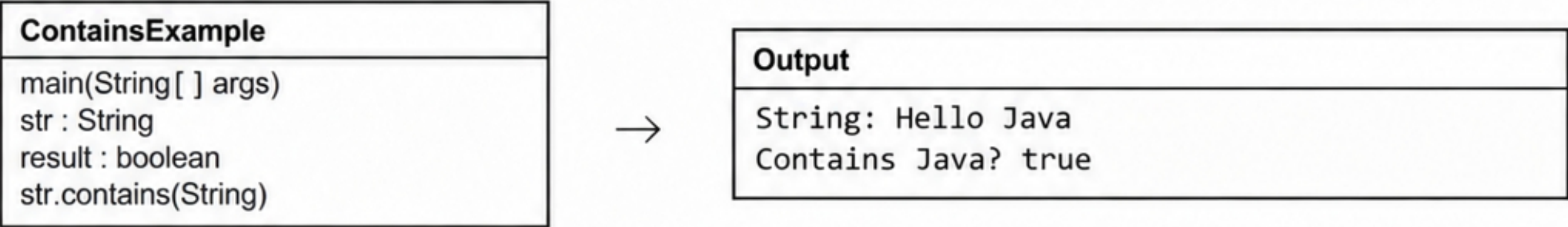
3. Check if the string contains "Java" using str.contains("Java").

4. Store the result in result.

5. Print the string and the result.

6. Stop.

UML Diagram



Program Code

```
class ContainsExample {  
    public static void main(String[] args) {  
        String str = "Hello Java";  
  
        boolean result = str.contains("Java");  
  
        System.out.println("String: " + str);  
        System.out.println("Contains Java? " + result);  
    }  
}
```

Code Explanation

Output Screenshot

1. `String str = "Hello Java";` creates a String variable containing the text "Hello Java".

2. `str.contains("Java")` checks whether the string contains the sequence "Java".

3. The method returns `true` because "Java" is present in the string.

4. `System.out.println()` displays the string and the result.

String: Hello Java
Contains Java? true

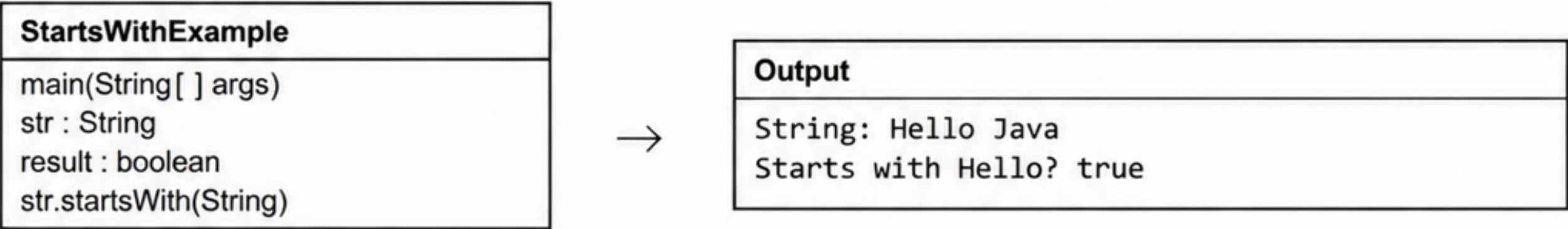
12. startsWith() — str

Objective
Objective: To understand and demonstrate the Java String method startsWith() using a simple example.

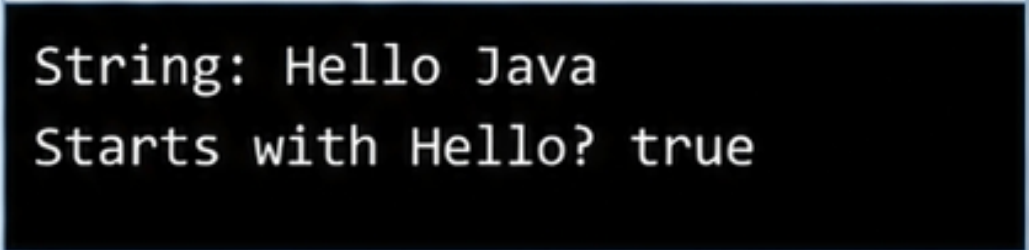
Task Description
Task Description: Write and execute the class StartsWithExample to demonstrate that startsWith() checks whether a string starts with the specified prefix.

Step-wise Execution	Algorithm / Logic
1. Create a String variable named str. 2. Store the value "Hello Java" in it. 3. Call the startsWith("Hello") method using str. 4. Store the returned value in a boolean variable result. 5. Display the string and whether it starts with "Hello".	1. Start. 2. Initialize str with "Hello Java". 3. Check if the string starts with "Hello" using str.startsWith("Hello"). 4. Store the result in result. 5. Print the string and the result. 6. Stop.

UML Diagram



Program Code
<pre>class StartsWithExample { public static void main(String[] args) { String str = "Hello Java"; boolean result = str.startsWith("Hello"); System.out.println("String: " + str); System.out.println("Starts with Hello? " + result); } }</pre>

Code Explanation	Output Screenshot
1. <code>`String str = "Hello Java";`</code> creates a String variable containing the text "Hello Java". 2. <code>`str.startsWith("Hello")`</code> checks whether the string starts with the specified prefix "Hello". 3. The method returns <code>`true`</code> because the string begins with "Hello". 4. <code>`System.out.println()`</code> displays the string and the result.	

13. endsWith() — str

Objective

Objective: To understand and demonstrate the Java String method **endsWith()** using a simple example.

Task Description

Task Description: Write and execute the class **EndsWithExample** to demonstrate that **endsWith()** checks whether a string ends with the specified suffix.

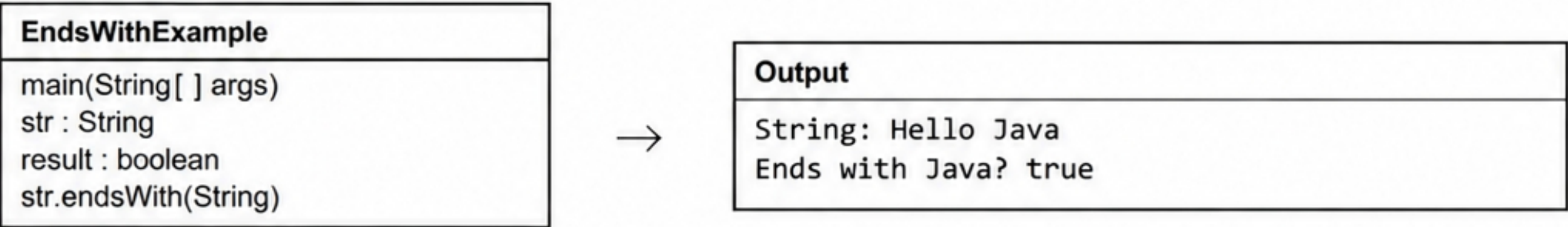
Step-wise Execution

1. Create a String variable named str.
2. Store the value "Hello Java" in it.
3. Call the endsWith("Java") method using str.
4. Store the returned value in a boolean variable result.
5. Display the string and whether it ends with "Java".

Algorithm / Logic

1. Start.
2. Initialize str with "Hello Java".
3. Check if the string ends with "Java" using str.endsWith("Java").
4. Store the result in result.
5. Print the string and the result.
6. Stop.

UML Diagram



Program Code

```
class EndsWithExample {  
    public static void main(String[] args) {  
        String str = "Hello Java";  
  
        boolean result = str.endsWith("Java");  
  
        System.out.println("String: " + str);  
        System.out.println("Ends with Java? " + result);  
    }  
}
```

Code Explanation

1. `String str = "Hello Java";` creates a String variable containing the text "Hello Java".
2. `str.endsWith("Java")` checks whether the string ends with the specified suffix "Java".
3. The method returns `true` because the string ends with "Java".
4. `System.out.println()` displays the string and the result.

Output Screenshot

```
String: Hello Java  
Ends with Java? true
```


14. replace() — str

Objective

Objective: To understand and demonstrate the Java String method **replace()** using a simple example.

Task Description

Task Description: Write and execute the class **ReplaceExample** to demonstrate that **replace()** replaces characters or a sequence of characters with another sequence.

Step-wise Execution

1. Create a String variable named str.

2. Store the value "Hello" in it.

3. Call the replace('l', 'x') method using str.

4. Store the returned value in a variable result.

5. Display the original string and the replaced string.

Algorithm / Logic

1. Start.

2. Initialize str with "Hello".

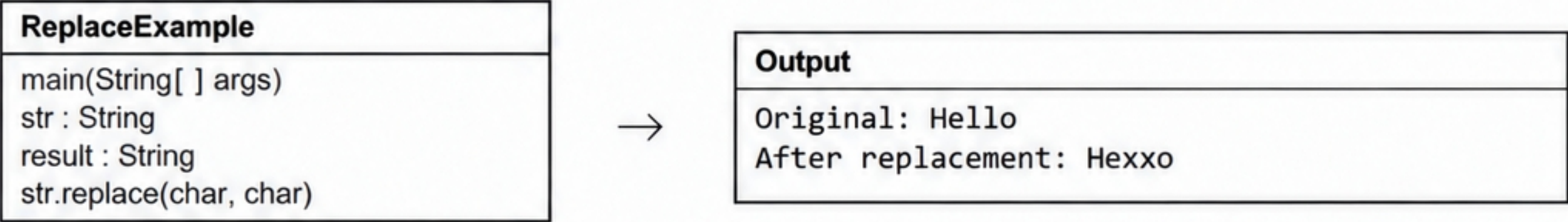
3. Replace character 'l' with 'x' using str.replace('l', 'x').

4. Store the result in result.

5. Print the original string and the replaced string.

6. Stop.

UML Diagram



Program Code

```
class ReplaceExample {  
    public static void main(String[] args) {  
        String str = "Hello";  
  
        String result = str.replace('l', 'x');  
  
        System.out.println("Original: " + str);  
        System.out.println("After replacement: " + result);  
    }  
}
```

Code Explanation

1. 'String str = "Hello";' creates a String variable containing the text "Hello".

2. 'str.replace('l', 'x')' replaces every occurrence of l with x.

3. The replaced string is stored in 'result'.

4. 'System.out.println()' displays the original string and the replaced string.

Output Screenshot

Original: Hello
After replacement: Hexxo

15. trim() — str

Objective

Objective: To understand and demonstrate the Java String method **trim()** using a simple example.

Task Description

Task Description: Write and execute the class **TrimExample** to demonstrate that **trim()** removes leading and trailing whitespace characters from a string.

Step-wise Execution

Algorithm / Logic

1. Create a String variable named str.

2. Store the value " Hello Java " in it (with leading and trailing spaces).

3. Call the trim() method using str.

4. Store the returned value in a variable result.

5. Display the original string and the trimmed string.

1. Start.

2. Initialize str with " Hello Java ".

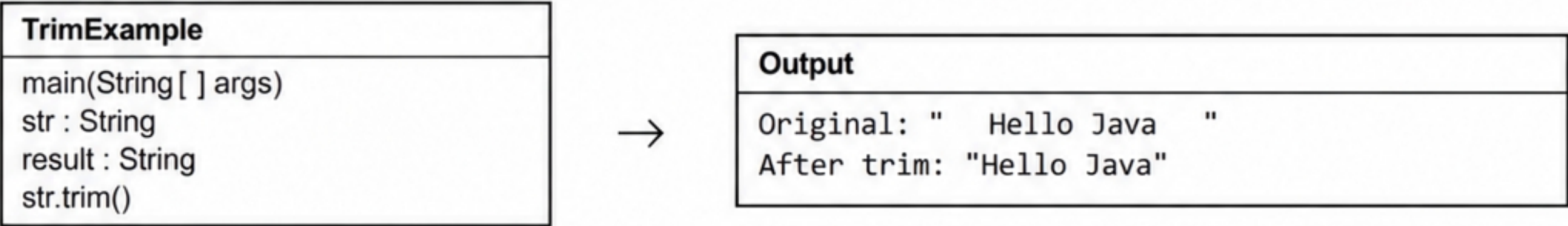
3. Apply trim() to str.

4. Store the result in result.

5. Print the original string and the trimmed string.

6. Stop.

UML Diagram



Program Code

```
class TrimExample {  
    public static void main(String[] args) {  
        String str = " Hello Java ";  
  
        String result = str.trim();  
  
        System.out.println("Original: \"" + str + "\"");  
        System.out.println("After trim: \"" + result + "\"");  
    }  
}
```

Code Explanation

Output Screenshot

1. `String str = " Hello Java ";` creates a String variable containing leading and trailing spaces.

2. `str.trim()` removes the leading and trailing whitespace characters.

3. The trimmed string is stored in `result`.

4. `System.out.println()` displays the original string (with spaces) and the trimmed string.

Original: " Hello Java "

After trim: "Hello Java"